

九联星闪开发板 Wi-Fi 接口与功能说明

适用平台：九联星闪开发板，基于海思 WS63E 芯片。

本文档汇总 WS63E 在 OpenHarmony (`nearlink_dk_3863`) 工程下的 **Wi-Fi 能力、软件接口、典型用法与仓库示例**，供课程实验与综合项目参考。

文档更新于 2026-05-21；第七节源码与 `applications/sample/wifi-iot/app/wifi_sdk_demo/` **保持同步**。

开发约定：课程与综合项目 Wi-Fi 联网应 **直接调用 SDK 的 `wifi_*` 接口**，实现方式见本文第七节完整示例。

课程标准实验：使用 **2 块**九联星闪开发板配合——一块为 **AP 板**，一块为 **STA 板**（见 2.3、7.12 节）。

一、概述

1.1 芯片与开发板

项目	说明
芯片	海思 WS63E ，片内集成 Wi-Fi （与 BLE、 SLE 星闪 等共存）
开发板	九联星闪开发板 ，板载 Wi-Fi 天线，无需外接 Wi-Fi 模块
工程产品	<code>nearlink_dk_3863</code> ，固件目标 <code>ws63-liteos-app</code>

1.2 Wi-Fi 能做什么

- STA（站点）**：连接对端开发板或其它 2.4G AP，获取 IP 后进行联调或上云。
- SoftAP（软热点）**：开发板自身发出热点，供 **另一块星闪开发板（STA）** 或 PC/手机连接。
- 上层协议**：在联网基础上使用 **TCP/UDP**（lwIP）、**MQTT**、**HTTP** 等（见厂商 demo 与 SDK 文档）。

1.3 与星闪（SLE）的分工

通信	典型场景
Wi-Fi	上云、远程监控、大流量； 双板实验 时 AP/STA 处于同一 <code>192.168.43.0/24</code> 网段
SLE	板间低时延近距、手机星闪生态（见 15-九联星闪开发板星闪SLE接口与功能说明.md ）

二者可同时存在于同一固件，但需注意 **RAM/任务栈** 与初始化顺序。

二、硬件与网络接口

2.1 硬件形态

- Wi-Fi 射频与基带在 **WS63E 片内**，通过板载 **天线** 辐射，**不占用** 排针 GPIO 作为 SDIO/SPI 外挂 Wi-Fi 模块。
- 烧录、调试用的 **USB 串口** 与 Wi-Fi 数据通路独立；联网后业务数据走无线，调试日志仍走 USB UART。

2.2 逻辑网络接口名

SDK / lwIP 中常见接口名（以厂商示例为准）：

模式	典型接口名	说明
STA	wlan0 等	关联 AP 后由 DHCP 或静态配置 IP
SoftAP	ap0	热点侧，常配 192.168.43.1/24 等网段

具体以 `netif_find()`、`wifi_linked_info` 中返回的 `ifname` 为准。

2.3 Wi-Fi 双板实验（课程标准做法）

本课程 **Wi-Fi STA/AP 实验** 使用 **2 块九联星闪开发板** 配合完成：一块为 **AP 板（SoftAP 热点）**，另一块为 **STA 板（站点）**。无需手机热点或实验室路由器。

板子	固件	串口预期
AP 板	第七节 <code>wifi_softap_demo.c</code>	<code>[WIFI_AP] started</code> 、 <code>[wifiSoftApDemo] event sta_join</code> 、 <code>online=1</code>
STA 板	第七节 <code>wifi_sta_demo.c</code>	<code>[WIFI_STA] IP 192.168.43.x</code> 、 <code>[wifiStaDemo] stats</code>

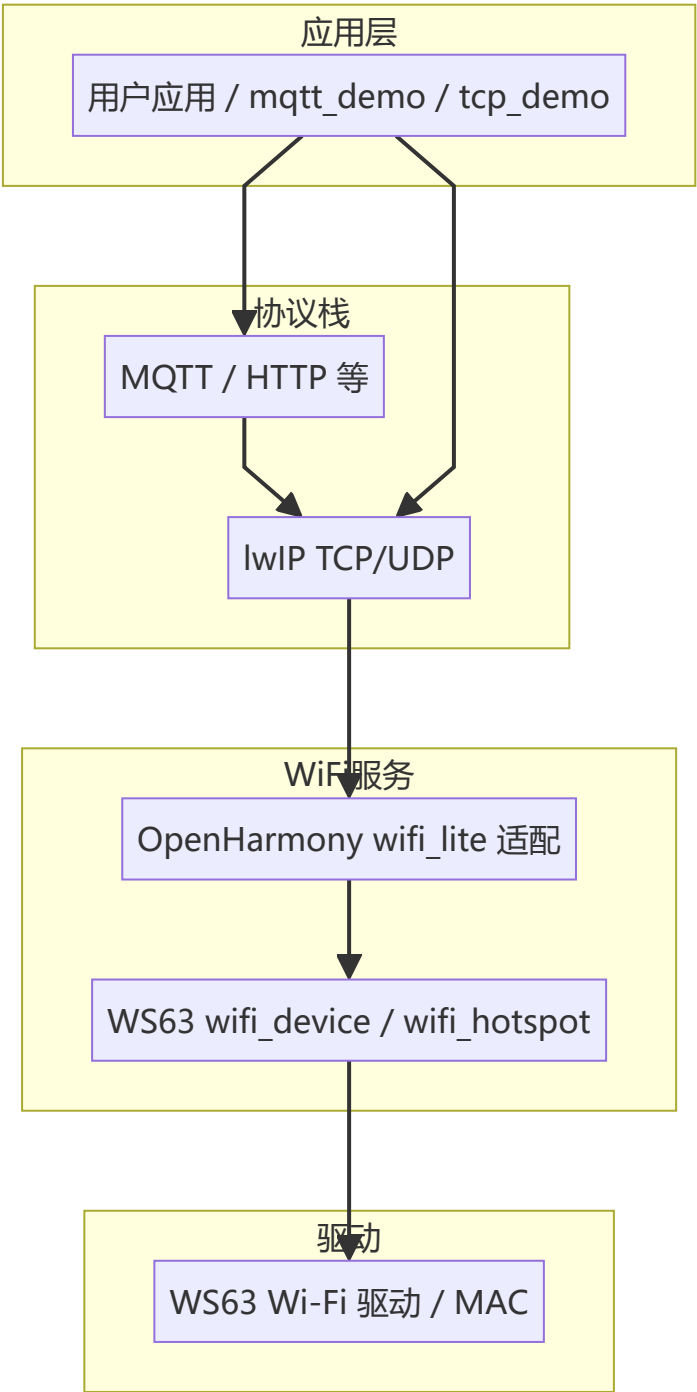
协作要点：

1. 两板 **SSID、密码、加密方式** 必须一致（示例默认 `OHOS_AP / 123456789`）。
2. 建议 **先烧录并上电 AP 板**，再启动 STA 板，便于 STA 扫描到热点。
3. 两板距离约 **0.5 ~ 1 m**，天线相对，避免金属遮挡。
4. 固定网段 `192.168.43.0/24`：AP 板 `192.168.43.1`，STA 板由 AP 侧 DHCP 分配（如 `192.168.43.2`）。
5. 联通验证：STA 板串口有 IP 后，AP 板每 5s 打印 `online=1`（`wifi_softap_get_sta_list`）；STA 板打印 `IPv4=192.168.43.x`。

完整源码与烧录步骤见本文 **第七节、7.12 节**。

WS63E 仅支持 **2.4GHz Wi-Fi**。若自行扩展为连接手机热点，手机须开 **2.4G** 个人热点（勿用仅 5GHz 热点）。

三、软件架构（分层）



层级	路径/模块（本仓）	作用
应用	applications/sample/wifi-iot/app/、vendor/.../ws63_sample/14~22_*	连接热点、MQTT 发布订阅等
lwIP	SDK 协议栈	IP、TCP、UDP、DHCP
Wi-Fi 中间件	device/soc/hisilicon/ws63v100/sdk/include/middleware/services/wifi/	STA/AP 配置、扫描、连接
OH 适配	device/soc/hisilicon/ws63v100/adapters/hals/communication/wifi_lite/wifiservice/	封装为 wifi_device.c、wifi_hotspot.c 等
事件	ws63_wifi_event.h 等	扫描完成、连接状态变化回调

应用开发时，**应直接调用 SDK 的 `wifi_*`**（见第七节完整示例）。完整 OpenHarmony 标准 Wi-Fi HAL 在 `wifi_lite` 适配层，课程轻量应用通常不必经过该层。

四、工作模式与功能

4.1 STA（站点）模式

功能：扫描 SSID → 选择 AP → 认证连接 → 获取 IP → 进行 socket 通信。

能力	说明
扫描	单次/高级扫描，获取 AP 列表（SSID、RSSI、信道等）
连接	WPA/WPA2-PSK 等（ <code>wifi_sta_config_stru</code> ）
重连	可配置自动重连策略 <code>wifi_sta_set_reconnect_policy</code>
状态	连接/断开事件回调；查询当前 AP 信息 <code>wifi_sta_get_ap_info</code>
省电	支持 PM 相关接口（如 <code>wifi_sta_set_pm</code> ）
快速连接	<code>wifi_sta_fast_connect</code> （保存过的 SSID 信息）

典型流程：

1. `wifi_init()`
2. `wifi_sta_enable()`
3. 注册 `wifi_event_stru`（扫描完成、连接变化）
4. `wifi_sta_scan()` → 解析结果 → `wifi_sta_connect(&config)`
5. 等待 `CONNECTED`，`netifapi` / DHCP 获取 IP
6. 创建 socket 或使用 MQTT 客户端

4.2 SoftAP（软热点）模式

功能：开发板作为小型 AP，其它设备连入后分配 IP（常内置 DHCP）。

能力	说明
开启热点	<code>wifi_softap_enable(&softap_config_stru)</code>
关闭热点	<code>wifi_softap_disable()</code>
参数	SSID、密码、 信道 （见 7.2.1）、加密方式（如 WPA2-PSK）
高级参数	Beacon 间隔、协议模式等（ <code>wifi_set_softap_config_advance</code> ）
已连 STA 列表	<code>wifi_softap_get_sta_list</code>
踢下线	<code>wifi_softap_deauth_sta</code>

SoftAP 双板实验常用网段（第七节示例采用）：

- AP IP: `192.168.43.1`
- 掩码: `255.255.255.0`
- 网关: `192.168.43.2`（示例配置）

4.3 上层网络功能（依赖 Wi-Fi 联网后）

功能	本仓示例	说明
TCP Client	15_tcpclient	lwIP socket 客户端
TCP Server	16_tcpserver	lwIP socket 服务端
UDP	17_udpcclient、18_udpserver	无连接数据报
MQTT	20_mqtt_demo、21_mqtt_led、22_mqtt_sensor、28_paho_mqtt	联网部分可改为第七节 STA 示例，再跑 MQTT
HTTP	官方《WS63V100 HTTP 开发指南》	文档树 applications/docs/官方文档/docs/software/

五、主要 API 接口

5.1 初始化与 STA（SDK）

头文件：
device/soc/hisilicon/ws63v100/sdk/include/middleware/services/wifi/wifi_device.h
(适配层对外亦有 ws63_wifi_device.h，接口对应。)

API	功能
wifi_init() / wifi_deinit()	Wi-Fi 协议栈初始化/去初始化
wifi_sta_enable() / wifi_sta_disable()	使能/关闭 STA 模式
wifi_sta_scan() / wifi_sta_scan_advance()	扫描周围 AP
wifi_sta_get_scan_info()	获取扫描结果
wifi_sta_connect()	连接指定 AP（SSID、密码等）
wifi_sta_disconnect()	断开连接
wifi_sta_get_ap_info()	当前连接 AP 信息
wifi_sta_set_reconnect_policy()	自动重连策略

5.2 SoftAP（SDK）

头文件：wifi_hotspot.h、wifi_hotspot_config.h 等。

API	功能
wifi_softap_enable()	按配置启动热点
wifi_softap_disable()	关闭热点

API	功能
wifi_softap_get_sta_list()	已连接终端列表
wifi_softap_set_mac_addr() / get	MAC 地址配置

5.3 事件回调

通过 `wifi_event_stru` 注册，常用：

- `wifi_event_scan_state_changed`：扫描结束
- `wifi_event_connection_changed`：关联成功/失败/断开（如打印 `NOTICE:CONNECTED`）

STA/AP 完整流程见 **第七节**；也可对照 `wifi_device.h`、`wifi_hotspot.h` 与 SDK 开发指南中的示例。

5.4 OpenHarmony 适配层 (wifi_lite)

路径：

`device/soc/hisilicon/ws63v100/adapter/hals/communication/wifi_lite/wifiservice/`

源文件	作用
wifi_device.c	STA 扫描、连接、配置管理
wifi_hotspot.c	SoftAP 启停、STA 列表
wifi_event.c	事件分发

课程 `wifi-iot` 轻量应用若未走完整 HDF Wi-Fi 服务，可直接参考 **SDK + 厂商 demo** 路径，上手更快。

5.5 lwIP 与 netif

联网成功后使用标准 lwIP 接口，例如：

- `netif_find(iframe)`
- `netifapi_netif_set_addr()`（AP 模式配 IP）
- `lwip_socket()` 等（TCP/UDP demo）

头文件在 SDK 中：`lwip/netifapi.h`、`lwip/sockets.h` 等。

六、本仓库示例工程索引

编号	目录 (vendor 镜像在 applications/docs/官方文档/vendor/.../demo/)	功能
15	15_tcpclient	TCP 客户端（需先完成第七节 STA 联网）
16	16_tcpserver	TCP 服务端

编号	目录 (vendor 镜像在 applications/docs/官方文档/vendor/.../demo/)	功能
17	17_udpclient	UDP 客户端
18	18_udpserver	UDP 服务端
20	20_mqtt_demo	MQTT 与 PC 通信
21	21_mqtt_led	MQTT 控制 LED
22	22_mqtt_sensor	MQTT 上报温湿度
28	28_paho_mqtt	paho MQTT 库示例

推荐学习顺序：本文 **第七节** (SDK 直连联网) → 20_mqtt_demo 或 15_tcpclient (业务，将联网改为自有 STA 模块)。

七、完整示例：直接调用 Wi-Fi SDK

本节给出 **双板 Wi-Fi 实验** 所需源码：**AP 板** 烧录 SoftAP 固件，**STA 板** 烧录 STA 固件。代码直接调用 wifi_* 与 lwIP，源码中含逐步中文注释，便于实验报告摘录。

7.0 wifi_sdk_demo 如何生成

wifi_sdk_demo 不会由编译器自动生成，也不能用 make_app.py -c 直接得到完整 Wi-Fi 工程（-c 仅创建 helloworld 式单文件模板）。课程标准做法如下。

方式一：使用仓库自带工程（推荐）

工程已放在：

applications/sample/wifi-iot/app/wifi_sdk_demo/

启用并编译（在源码根目录）：

```
# 1. 启用工程（写入 app/BUILD.gn、config.py、ohos.cmake）
python3 applications/make_app.py -e wifi_sdk_demo wifi_sdk_demo

# 2. 编译（可先 -x 禁用其它 demo，避免固件过大）
python3 applications/make_app.py -x pwm_servo pwm_servo_demo # 示例：禁用舵机 demo
python3 applications/make_app.py -b
```

固件路径：out/nearlink_dk_3863/nearlink_dk_3863/ws63-liteos-app/ws63-liteos-app_all.fwpkg

AP 板 / STA 板分别烧录：两板不能烧录同一份固件，须各改一次 BUILD.gn 的 sources 后 分别编译、分别烧录（见下表）。

板子	修改 <code>wifi_sdk_demo/BUILD.gn</code> 的 sources	再执行
STA 板（默认）	保留 <code>wifi_sta_demo.c</code> ，注释 <code>wifi_softap_demo.c</code>	<code>make_app.py -b</code> → 烧录
AP 板	注释 <code>wifi_sta_demo.c</code> ，保留 <code>wifi_softap_demo.c</code>	<code>make_app.py -b</code> → 烧录

查看已启用应用：`python3 applications/make_app.py -l`

方式二：从零手动创建

- 在 `applications/sample/wifi-iot/app/` 下新建目录 `wifi_sdk_demo`。
- 将本节 7.3~7.10 的源码保存为对应文件名（见 7.1 目录树），`BUILD.gn` 用 7.11 内容。
- 按 [04-创建新项目的方法.md](#) 注册组件，或执行：
`python3 applications/make_app.py -e wifi_sdk_demo wifi_sdk_demo`
- 按上表切换 AP/STA 入口后编译烧录。

方式三：仅用 `make_app.py` 建空壳

```
python3 applications/make_app.py -c wifi_sdk_demo wifi_sdk_demo
```

会生成 `wifi_sdk_demo.c` + 简易 `BUILD.gn`，仍需删除模板 `.c`，并拷入本节 Wi-Fi 源文件、替换为 7.11 的 `BUILD.gn`。

7.1 工程目录建议

```
applications/sample/wifi-iot/app/wifi_sdk_demo/
├─ wifi_sdk_common.c/h      # wifisdkEnsureInit（避免重复 wifi_init 死循环）
├─ wifi_sta_sdk.h           # STA 对外接口
├─ wifi_sta_sdk.c           # STA: 扫描、连接、DHCP
├─ wifi_softap_sdk.h        # SoftAP 对外接口
├─ wifi_softap_sdk.c        # SoftAP: 开热点、配 IP、DHCP Server
├─ wifi_tcp_common.h        # TCP 端口、AP IP 等公共宏
├─ wifi_tcp_server.c/h      # AP 板 TCP Server（lwIP socket）
├─ wifi_tcp_client.c/h      # STA 板 TCP Client
├─ wifi_sta_demo.c          # STA 入口（烧录用）
├─ wifi_softap_demo.c       # SoftAP 入口（另一块板烧录用）
└─ BUILD.gn
```

板子	编译入口	说明
AP 板	<code>wifi_softap_demo.c</code>	<code>BUILD.gn</code> 中注释掉 <code>wifi_sta_demo.c</code>
STA 板	<code>wifi_sta_demo.c</code>	<code>BUILD.gn</code> 中注释掉 <code>wifi_softap_demo.c</code>

两板 SSID/密码须完全一致（默认 `OHOS_AP / 123456789`）。STA 板宏 `WIFI_STA_SSID`、`WIFI_STA_PSK` 与 AP 板 `WIFI_AP_SSID`、`WIFI_AP_PSK` 相同。

代码分层（维护时勿混淆）：

层次	文件	职责
公共	wifi_sdk_common.c	wifiSdkEnsureInit()，避免重复 wifi_init 死循环
SDK 封装	wifi_sta_sdk.c、 wifi_softap_sdk.c	扫描/连接/DHCP 或开热点/DHCPs， 不含 业务事件统计
应用入口	wifi_sta_demo.c 或 wifi_softap_demo.c	wifi_register_event_cb、周期打印统计， 二选一编译

7.2 公共说明

项目	说明
头文件路径	device/soc/hisilicon/ws63v100/sdk/include/middleware/services/wifi/
lwIP	lwip/netifapi.h、lwip/nettool/misc.h（dhcp_clients_info_show）
STA 网口名	wlan0
SoftAP 网口名	ap0
加密类型	双板实验使用 WIFI_SEC_TYPE_WPA2PSK（枚举值 2）
工作频段	WS63E 板载 Wi-Fi 为 2.4GHz
扫描等待	STA 板 wifi_sta_scan() 后默认等待 5s （WIFI_STA_SCAN_WAIT_MS），扫不到 AP 板时可改为 8~10s
初始化	固件启动时常已 wifi_init；示例用 wifiSdkEnsureInit()，勿对 wifi_init() 失败死循环重试
事件监听	STA 板： wifi_event_connection_changed、wifi_event_scan_state_changed（见 7.9）
事件监听	AP 板： wifi_event_softap_sta_join/leave + wifi_softap_get_sta_list（见 7.10）
AP 网关字段	netif 网关 192.168.43.2 （与 SDK softap_sample 一致）；AP 接口 IP 仍为 192.168.43.1
TCP 联调	AP 板 wifiTcpServerStartThread(8888)；STA 板连 192.168.43.1:8888（wifi_tcp_*.c）

7.2.1 SoftAP 工作信道（WIFI_AP_CHANNEL / channel_num）

wifi_softap_demo.c 中的 WIFI_AP_CHANNEL 对应 SDK 结构体 softap_config_stru.channel_num，表示 **2.4GHz 频段下的 Wi-Fi 信道号**（不是频率 MHz），经 wifi_softap_enable() 下发给协议栈。

SDK 校验依据（本仓库源码）

依据	位置	说明
配置范围	<code>soc_wifi_service_api.c</code> → <code>wifi_check_softap_config()</code>	注释「2.4g 信道号最大 14」； <code>channel_num</code> 须在 0~14 ，否则配置失败
实际启用	<code>wifi_api.c</code> → <code>wifi_channel_check()</code>	未开 <code>CONFIG_ACS</code> 时要求 chan ≥ 1 ； 0 不能作为工作信道
2.4G 上限宏	<code>wifi_api.h</code> → <code>WPA_24G_CHANNEL_NUMS</code>	值为 14
国家码收紧	<code>wifi_api.c</code> → <code>wifi_channel_check()</code> + <code>wifi_api.h</code>	见下表

按国家/地区码允许的最大信道（`wifi_channel_check`）

国家码（SDK）	最大信道	典型区域
US	11	美国等
JP、ZZ	14	日本等（14 信道仅在少数地区合法）
其它（中国大陆多数情况落在此类）	13	实验室请使用 1~13 ， 不要用 14

信道号与中心频率（IEEE 802.11 2.4GHz）

中心频率（MHz）：(f = 2407 + 5 \times n) ((n) 为信道号)。

信道号 (n)	中心频率 (MHz)	备注
1	2412	常用
6	2437	推荐 （20MHz 带宽下与 1、11 互不重叠）
7	2442	本仓库示例默认值 （厂商 <code>14_easy_wifi</code> 亦用 7）
11	2462	推荐
13	2472	国内常见上限

20MHz 带宽时，经典 **互不重叠**信道为 **1、6、11**。信道 **7** 与 6、8 邻频，多台 AP 并存时易干扰；**单 AP + 单 STA 双板实验** 用 7 通常可行。

实验选型建议

场景	建议 <code>WIFI_AP_CHANNEL</code>
双板 AP+STA，周围 Wi-Fi 少	6 或 7 （与示例一致）
教室多组 AP 板同时开热点	各组错开： 1 / 6 / 11 （或 1、6、13）
中国大陆	1~13 ；避免 14

修改示例（`wifi_softap_demo.c`）：

```
#define WIFI_AP_CHANNEL 6    /* 1~13（中国）；须通过 SDK wifi_channel_check */
```

写实验报告可引用的依据：① 海思 SDK `wifi_check_softap_config` / `wifi_channel_check`；② IEEE 802.11 2.4GHz 信道定义；③ 国内 2.4GHz Wi-Fi 设备常规使用 **1~13 信道**；④ 多 AP 部署惯例采用 **1、6、11** 降低同频干扰。

7.3 公共模块 `wifi_sdk_common.h`

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_sdk_common.h` 同步（文档更新时以仓库为准）。

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_sdk_common.h - STA/AP SDK 共用的 Wi-Fi 初始化辅助
 */
#ifndef WIFI_SDK_COMMON_H
#define WIFI_SDK_COMMON_H

/**
 * @brief 保证 Wi-Fi 协议栈可用
 * @param log_tag 串口日志前缀，如 "[WIFI_STA]"
 *
 * 若系统启动阶段已调用 wifi_init()，再次调用会失败并打印
 * “WiFi has been initialized”，本函数不循环重试，避免死锁。
 */
void WifiSdkEnsureInit(const char *log_tag);

#endif
```

7.4 公共模块 `wifi_sdk_common.c`

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_sdk_common.c` 同步（文档更新时以仓库为准）。

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_sdk_common.c - 处理“系统已初始化 Wi-Fi”与“应用首次初始化”两种情况
 */
#include "wifi_sdk_common.h"
#include <stdio.h>
#include "errcode.h"
#include "wifi_device.h"

void WifiSdkEnsureInit(const char *log_tag)
{
    if (wifi_init() == ERRCODE_SUCC) {
        printf("%s wifi_init ok\r\n", log_tag);
        return;
    }
}
```

```

}
/*
 * nearlink_dk_3863 启动流程中协议栈常已 wifi_init,
 * 再次调用会打印 Srv:xxx:WiFi has been initialized 并返回 FAIL, 属正常。
 */
printf("%s skip wifi_init (already initialized by system)\r\n", log_tag);
}

```

7.5 STA 模块头文件 `wifi_sta_sdk.h`

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_sta_sdk.h` 同步（文档更新时以仓库为准）。

```

/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_sta_sdk.h - STA 板连接 AP 与 DHCP 接口（无事件监听）
 */
#ifndef WIFI_STA_SDK_H
#define WIFI_STA_SDK_H

#include "errcode.h"

/**
 * @brief 扫描、关联指定 SSID 并在 wlan0 上 DHCP 获取 IP
 * @param ssid 须与 AP 板热点名一致（默认 OHOS_AP）
 * @param psk 须与 AP 板密码一致
 */
errcode_t wifiStaConnectAp(const char *ssid, const char *psk);

/** @brief 停止 DHCP 并 wifi_sta_disconnect */
void wifiStaDisconnectAp(void);

#endif

```

7.6 STA 模块实现 `wifi_sta_sdk.c`

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_sta_sdk.c` 同步（文档更新时以仓库为准）。

```

/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_sta_sdk.c - STA 模式连接 AP 与 DHCP 客户端封装
 *
 * 流程概览：
 *   wifiSdkEnsureInit → wifi_sta_enable
 *   → [wifi_sta_scan → 等待 → 匹配 SSID → wifi_sta_connect → 等 WIFI_CONNECTED] ×
 *   重试
 *   → netifapi_dhcp_start(wlan0) → 等待 bound 与 IPv4
 *
 * 不包含 wifi_register_event_cb；连接过程事件请在 wifi_sta_demo.c 注册。

```

```

*/
#include "wifi_sta_sdk.h"
#include "wifi_sdk_common.h"
#include <stdio.h>
#include <string.h>
#include "securec.h"
#include "soc_osal.h"
#include "errcode.h"
#include "wifi_device.h"          /* wifi_init / wifi_sta_* */
#include "wifi_device_config.h"   /* wifi_sta_config_stru、DHCP、WIFI_MAX_KEY_LEN
*/
#include "wifi_event.h"           /* 可选: wifi_register_event_cb 扫描完成通知 */
#include "wifi_linked_info.h"     /* WIFI_CONNECTED、wifi_sta_get_ap_info */
#include "lwip/netifapi.h"        /* netifapi_dhcp_start 等 */
#include "lwip/nettool/misc.h"    /* dhcp_clients_info_show 调试打印 */

/* 串口日志前缀，便于过滤 */
#define WIFI_STA_LOG              "[WIFI_STA]"
/* STA 模式在 lwIP 中的网口名，关联 AP 后用于 DHCP */
#define WIFI_STA_IFNAME          "wlan0"
#define WIFI_MAX_SSID_LEN        33  /* SSID 最大长度（含结束符空间） */
#define WIFI_MAC_LEN             6
#define WIFI_SCAN_AP_LIMIT       64  /* 单次扫描最多缓存的 AP 数量 */
#define WIFI_STA_RETRY_MAX       5    /* 扫描+连接整轮最大重试次数 */
#define WIFI_STA_SCAN_WAIT_MS    5000 /* 发起扫描后等待时间(ms)，过短可能读不到 AP 板热点
*/
#define WIFI_CONN_POLL_TIMES     10   /* 轮询关联状态次数，每次间隔 500ms */
#define WIFI_DHCP_BOUND_TIMES    10   /* 等待 DHCP 绑定次数 */
#define WIFI_STA_IP_POLL_TIMES   50   /* 等待 IPv4 地址非 0，每次 100ms */

/* 保存 wlan0 的 netif 指针，断开时用于 dhcp_stop */
static struct netif *g_sta_netif = NULL;

/*
 * 从最近一次扫描结果中查找与 ssid 完全匹配的 AP，
 * 并填充 wifi_sta_connect() 所需的 cfg（含 BSSID、加密类型、密码）。
 */
static errcode_t WifiStaFindTargetAp(const char *ssid, const char *psk,
wifi_sta_config_stru *out_cfg)
{
    uint32_t num = WIFI_SCAN_AP_LIMIT;
    uint32_t scan_len = sizeof(wifi_scan_info_stru) * num;
    wifi_scan_info_stru *result = osal_kmalloc(scan_len, OSAL_GFP_ATOMIC);

    if (result == NULL) {
        return ERRCODE_MALLOC;
    }
    (void)memset_s(result, scan_len, 0, scan_len);

    /* num 入参为缓冲区容量，出参为实际 AP 个数 */
    if (wifi_sta_get_scan_info(result, &num) != ERRCODE_SUCC) {
        osal_kfree(result);
        return ERRCODE_FAIL;
    }

    for (uint32_t i = 0; i < num; i++) {

```

```

/* 先比长度再比内容, 减少误匹配 */
if (strlen(ssid) != strlen((const char *)result[i].ssid)) {
    continue;
}
if (memcmp(ssid, result[i].ssid, strlen(ssid)) != 0) {
    continue;
}
/* 从扫描条目拷贝 SSID、BSSID; 加密类型用 AP 广播的值 */
if (memcpy_s(out_cfg->ssid, WIFI_MAX_SSID_LEN, result[i].ssid,
WIFI_MAX_SSID_LEN) != EOK) {
    osal_kfree(result);
    return ERRCODE_MEMCPY;
}
if (memcpy_s(out_cfg->bssid, WIFI_MAC_LEN, result[i].bssid, WIFI_MAC_LEN)
!= EOK) {
    osal_kfree(result);
    return ERRCODE_MEMCPY;
}
out_cfg->security_type = result[i].security_type;
if (memcpy_s(out_cfg->pre_shared_key, WIFI_MAX_KEY_LEN, psk, strlen(psk))
!= EOK) {
    osal_kfree(result);
    return ERRCODE_MEMCPY;
}
out_cfg->ip_type = DHCP; /* 由 AP 板 DHCP Server 分配 192.168.43.x */
osal_kfree(result);
return ERRCODE_SUCC;
}

osal_kfree(result);
printf("%s target SSID not found: %s\r\n", WIFI_STA_LOG, ssid);
return ERRCODE_FAIL;
}

/*
 * wifi_sta_connect() 返回成功后, 链路层关联可能仍在进行,
 * 通过 wifi_sta_get_ap_info() 轮询直到 conn_state == WIFI_CONNECTED.
 */
static errcode_t wifistawaitConnected(void)
{
    wifi_linked_info_stru status;

    for (uint8_t i = 0; i < WIFI_CONN_POLL_TIMES; i++) {
        osal_msleep(500);
        (void)memset_s(&status, sizeof(status), 0, sizeof(status));
        if (wifi_sta_get_ap_info(&status) != ERRCODE_SUCC) {
            continue;
        }
        if (status.conn_state == WIFI_CONNECTED) {
            printf("%s associated with AP\r\n", WIFI_STA_LOG);
            return ERRCODE_SUCC;
        }
    }
    return ERRCODE_FAIL;
}

```

```

/*
 * 在 wlan0 上启动 DHCP 客户端，等待获取 IPv4。
 * 双板实验下 STA 板 IP 一般为 192.168.43.2 等，以串口打印为准。
 */
static errcode_t wifiStaStartDhcp(void)
{
    g_sta_netif = netifapi_netif_find(WIFI_STA_IFNAME);
    if (g_sta_netif == NULL) {
        printf("%s netif %s not found\r\n", WIFI_STA_LOG, WIFI_STA_IFNAME);
        return ERRCODE_FAIL;
    }

    if (netifapi_dhcp_start(g_sta_netif) != ERR_OK) {
        printf("%s dhcp_start fail lwip_err\r\n", WIFI_STA_LOG);
        return ERRCODE_FAIL;
    }

    /* 等待 DHCP 状态机进入 bound */
    for (uint8_t i = 0; i < WIFI_DHCP_BOUND_TIMES; i++) {
        osal_msleep(500);
        if (netifapi_dhcp_is_bound(g_sta_netif) == ERR_OK) {
            break;
        }
    }

    /* lwIP 中 IPv4 为小端存储，打印时需按字节拆解 */
    for (uint8_t i = 0; i < WIFI_STA_IP_POLL_TIMES; i++) {
        osal_msleep(100);
        if (g_sta_netif->ip_addr.u_addr.ip4.addr != 0) {
            uint32_t ip = g_sta_netif->ip_addr.u_addr.ip4.addr;
            printf("%s IP %u.%u.%u.%u\r\n", WIFI_STA_LOG,
                (ip & 0xff), (ip >> 8) & 0xff, (ip >> 16) & 0xff, (ip >> 24) &
0xff);
            (void)netifapi_netif_common(g_sta_netif, dhcp_clients_info_show,
NULL);
            return ERRCODE_SUCC;
        }
    }

    printf("%s DHCP timeout\r\n", WIFI_STA_LOG);
    return ERRCODE_FAIL;
}

/**
 * @brief 连接指定热点并获取 IPv4（双板实验 STA 板主流程）
 * @param ssid 目标 SSID，须与 AP 板一致
 * @param psk WPA 密码，须与 AP 板一致
 * @return ERRCODE_SUCC 已关联且 DHCP 拿到 IP；否则重试耗尽后 ERRCODE_FAIL
 */
errcode_t wifiStaConnectAp(const char *ssid, const char *psk)
{
    wifi_sta_config_stru cfg = {0};

    /* 步骤1：协议栈：系统可能已初始化，勿在 FAIL 时死循环重试 wifi_init */
    wifisdkEnsureInit(WIFI_STA_LOG);

```

```

/* 步骤2: 使能 STA 模式 (站点, 用于连接外部 AP) */
while (wifi_sta_enable() != ERRCODE_SUCC) {
    printf("%s sta_enable retry\r\n", WIFI_STA_LOG);
    osal_msleep(100);
}

/* 步骤3: 扫描 → 匹配 → 连接 → DHCP, 失败则进入下一轮重试 */
for (uint8_t retry = 0; retry < WIFI_STA_RETRY_MAX; retry++) {
    printf("%s scan round %u\r\n", WIFI_STA_LOG, retry);

    if (wifi_sta_scan() != ERRCODE_SUCC) {
        osal_msleep(1000);
        continue;
    }
    /*
     * 必须等待扫描完成再调用 wifi_sta_get_scan_info。
     * 进阶: 可注册 wifi_event_scan_state_changed, 在回调里置标志再读表, 无需固定延
    时。
     */
    osal_msleep(WIFI_STA_SCAN_WAIT_MS);

    if (wifiStaFindTargetAp(ssid, psk, &cfg) != ERRCODE_SUCC) {
        continue; /* 未扫到 AP 板: AP 未上电、等待时间过短、SSID/密码不一致 */
    }

    if (wifi_sta_connect(&cfg) != ERRCODE_SUCC) {
        printf("%s wifi_sta_connect fail\r\n", WIFI_STA_LOG);
        continue;
    }

    if (wifiStaWaitConnected() != ERRCODE_SUCC) {
        continue; /* 密码错误或信号弱可能导致关联超时 */
    }

    if (wifiStaStartDhcp() == ERRCODE_SUCC) {
        printf("%s connect success\r\n", WIFI_STA_LOG);
        return ERRCODE_SUCC;
    }
}

printf("%s connect failed after %u retries\r\n", WIFI_STA_LOG,
WIFI_STA_RETRY_MAX);
return ERRCODE_FAIL;
}

/**
 * @brief 断开当前 AP 并释放 wlan0 DHCP 租约
 */
void wifiStaDisconnectAp(void)
{
    if (g_sta_netif != NULL) {
        (void)netifapi_dhcp_stop(g_sta_netif);
        g_sta_netif = NULL;
    }
    (void)wifi_sta_disconnect();
    printf("%s disconnected\r\n", WIFI_STA_LOG);
}

```



```
}
```

7.7 SoftAP 模块头文件 `wifi_softap_sdk.h`

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_softap_sdk.h` 同步（文档更新时以仓库为准）。

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_softap_sdk.h - AP 板热点启动/停止接口（无事件监听）
 */
#ifndef WIFI_SOFTAP_SDK_H
#define WIFI_SOFTAP_SDK_H

#include "errcode.h"
#include "wifi_device_config.h"

/**
 * @brief 启动 SoftAP: enable、配置 ap0、启动 DHCP
 * @return ERRCODE_SUCC 成功；失败时已尝试 wifi_softap_disable
 */
errcode_t WifiSoftApStart(const char *ssid, const char *psk,
                          wifi_security_enum sec_type, uint8_t channel);

/** @brief 关闭热点并停止 ap0 上的 DHCP */
void WifiSoftApStop(void);

#endif
```

7.8 SoftAP 模块实现 `wifi_softap_sdk.c`

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_softap_sdk.c` 同步（文档更新时以仓库为准）。

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_softap_sdk.c - SoftAP（热点）能力封装
 *
 * 流程: WifiSdkEnsureInit → wifi_softap_enable → 等待 ap0 → 静态 IP
 *       → 停止其它网口 DHCP → netifapi_dhcps_start（为 STA 分配 192.168.43.x）
 *
 * 不包含 STA 上下线监听；事件与统计由 wifi_softap_demo.c 注册。
 */
#include "wifi_softap_sdk.h"
#include "wifi_sdk_common.h"
#include <stdio.h>
#include <string.h>
#include "securec.h"
#include "soc_osal.h"
#include "errcode.h"
```

```

#include "wifi_device.h"
#include "wifi_hotspot.h"
#include "wifi_hotspot_config.h"
#include "wifi_event.h"
#include "lwip/err.h"
#include "lwip/netif.h"
#include "lwip/netifapi.h"
#if LWIP_DHCP
#include "lwip/dhcp.h"
#endif

#define WIFI_AP_LOG "[WIFI_AP]"
#define WIFI_AP_IFNAME "ap0" /* SoftAP 在 lwIP 中的网口名 */
#define WIFI_AP_NETIF_WAIT_MS 100 /* 轮询 ap0 间隔 */
#define WIFI_AP_NETIF_RETRY 50 /* 最多等待约 5s */
#define WIFI_AP_SOFTAP_RETRY 20 /* wifi_softap_enable 重试次数 */
#define WIFI_AP_AFTER_ENABLE_MS 1000 /* enable 后等待驱动创建 netif */

static struct netif *g_ap_netif = NULL;

/**
 * @brief 停止除 ap0 外所有网口上的 DHCP Server
 * @note lwIP DHCP 绑定 UDP 67，全局通常只能有一个 Server，否则 dhcp_start 返回
ERR_MEM(-1)
 */
static void wifiApStopDhcpOnOtherNetifs(struct netif *ap_netif)
{
    struct netif *n = NULL;

    for (n = netif_list; n != NULL; n = n->next) {
        if (n == ap_netif) {
            continue;
        }
        (void)netifapi_dhcp_stop(n);
    }
}

/**
 * @brief 注册空事件表，避免部分固件打印“cd is not init”类告警
 * 业务回调请在 wifi_softap_demo.c 中单独 register
 */
static void wifiApRegisterEventStub(void)
{
    static wifi_event_stru s_ev = {0};
    static uint8_t s_registered;

    if (s_registered) {
        return;
    }
    if (wifi_register_event_cb(&s_ev) == ERRCODE_SUCC) {
        s_registered = 1;
    }
}

/**
 * @brief 轮询 netif_find("ap0") 直到驱动创建 SoftAP 网口

```

```

*/
static struct netif *wifiApwaitNetif(void)
{
    for (uint8_t i = 0; i < WIFI_AP_NETIF_RETRY; i++) {
        struct netif *netif = netif_find(WIFI_AP_IFNAME);
        if (netif != NULL) {
            return netif;
        }
        osal_msleep(WIFI_AP_NETIF_WAIT_MS);
    }
    return NULL;
}

/**
 * @brief 启动 SoftAP 并完成 ap0 地址与 DHCP Server
 * @param ssid      热点名称
 * @param psk       WPA2 密码
 * @param sec_type   一般为 WIFI_SEC_TYPE_WPA2PSK
 * @param channel    2.4G 信道号 (SDK 常用 0~14, 见文档 7.2.1)
 */
errcode_t wifiSoftApStart(const char *ssid, const char *psk,
    wifi_security_enum sec_type, uint8_t channel)
{
    softap_config_stru hapd = {0};
    ip4_addr_t ip = {0};
    ip4_addr_t mask = {0};
    ip4_addr_t gw = {0};
    err_t lwip_ret;

    while (wifi_is_wifi_initiated() == 0) {
        osal_msleep(WIFI_AP_NETIF_WAIT_MS);
    }

    wifiSdkEnsureInit(WIFI_AP_LOG);
    wifiApRegisterEventStub();

    if (memcpy_s(hapd.ssid, sizeof(hapd.ssid), ssid, strlen(ssid)) != EOK) {
        return ERRCODE_MEMCPY;
    }
    if (memcpy_s(hapd.pre_shared_key, WIFI_MAX_KEY_LEN, psk, strlen(psk)) != EOK)
{
        return ERRCODE_MEMCPY;
    }
    hapd.security_type = sec_type;
    hapd.channel_num = channel;

    for (uint8_t retry = 0; retry < WIFI_AP_SOFTAP_RETRY; retry++) {
        if (wifi_softap_enable(&hapd) == ERRCODE_SUCC) {
            break;
        }
        printf("%s softap_enable retry %u\r\n", WIFI_AP_LOG, retry);
        osal_msleep(200);
        if (retry == WIFI_AP_SOFTAP_RETRY - 1) {
            return ERRCODE_FAIL;
        }
    }
}

```

```

osal_msleep(WIFI_AP_AFTER_ENABLE_MS);
g_ap_netif = wifiApWaitNetif();
if (g_ap_netif == NULL) {
    printf("%s netif %s not found\r\n", WIFI_AP_LOG, WIFI_AP_IFNAME);
    (void)wifi_softap_disable();
    return ERRCODE_FAIL;
}

/* 与厂商 softap_sample 一致: 接口 IP=.1, 网关字段=.2, STA 由 DHCP 分配 .x */
IP4_ADDR(&ip, 192, 168, 43, 1);
IP4_ADDR(&mask, 255, 255, 255, 0);
IP4_ADDR(&gw, 192, 168, 43, 2);

lwip_ret = netifapi_netif_set_addr(g_ap_netif, &ip, &mask, &gw);
if (lwip_ret != ERR_OK) {
    printf("%s set_addr fail lwip_err=%d\r\n", WIFI_AP_LOG, (int)lwip_ret);
    (void)wifi_softap_disable();
    return ERRCODE_FAIL;
}

wifiApStopDhcpsonOtherNetifs(g_ap_netif);
(void)netifapi_dhcpstop(g_ap_netif);

#if LWIP_DHCP
    if (g_ap_netif->dhcp != NULL) {
        printf("%s dhcp already running on ap0\r\n", WIFI_AP_LOG);
    } else
#endif
{
    lwip_ret = netifapi_dhcpstart(g_ap_netif, NULL, 0);
    if (lwip_ret != ERR_OK) {
        printf("%s dhcpstart fail lwip_err=%d (ERR_MEM=-1 多为67端口占
用)\r\n",
            WIFI_AP_LOG, (int)lwip_ret);
        (void)wifi_softap_disable();
        return ERRCODE_FAIL;
    }
}

printf("%s started SSID=%s channel=%u IP=192.168.43.1\r\n",
    WIFI_AP_LOG, ssid, channel);
return ERRCODE_SUCC;
}

/**
 * @brief 停止 DHCP 并关闭 SoftAP
 */
void wifiSoftApStop(void)
{
    if (g_ap_netif != NULL) {
        (void)netifapi_dhcpstop(g_ap_netif);
        g_ap_netif = NULL;
    }
    (void)wifi_softap_disable();
    printf("%s stopped\r\n", WIFI_AP_LOG);
}

```

```
}
```

7.9 STA 入口 `wifi_sta_demo.c` (STA 板烧录)

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_sta_demo.c` 同步 (文档更新时以仓库为准)。

```
/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_sta_demo.c - 双板实验 **STA 板** 应用入口
 *
 * 职责：
 *   1. 注册 STA 模式事件（连接状态变化、扫描完成）；
 *   2. 调用 wifi_sta_sdk 的 wifiStaConnectAp 连接 AP 板热点；
 *   3. 周期查询 wifi_sta_get_ap_info 与 wlan0 IPv4，打印连接统计。
 *
 * 烧录：BUILD.gn 只保留本文件，注释 wifi_softap_demo.c。
 * 事件与统计放在 demo，不放在 wifi_sta_sdk.c。
 */
#include <stdio.h>
#include <string.h>
#include "ohos_init.h"
#include "cmsis_os2.h"
#include "errcode.h"
#include "wifi_device.h"
#include "wifi_event.h"
#include "wifi_linked_info.h"
#include "wifi_sta_sdk.h"
#include "lwip/netif.h"

/* 须与 AP 板 wifi_softap_demo.c 中 WIFI_AP_SSID / WIFI_AP_PSK 完全一致 */
#define WIFI_STA_SSID "OHOS_AP"
#define WIFI_STA_PSK "123456789"
/* STA 关联后 lwIP 网口名，用于读取 DHCP 获得的 IP */
#define WIFI_STA_IFNAME "wlan0"
#define WIFI_STA_STATS_PERIOD_MS 5000

/**
 * @brief STA 板侧连接过程累计计数（供实验观察重连、扫描次数）
 */
typedef struct {
    uint32_t scan_done; /* wifi_event_scan_state_changed 次数 */
    uint32_t connect_ok; /* 连接可用（WIFI_STATE_AVALIABLE）次数 */
    uint32_t disconnect; /* 断开（WIFI_STATE_NOT_AVALIABLE）次数 */
} WifiStaConnStats;

static WifiStaConnStats g_sta_stats;
static uint8_t g_sta_event_registered;

/**
 * @brief 将 wifi_conn_state_enum 转为可读字符串
 */
static const char *WifiStaConnStateStr(wifi_conn_state_enum state)
```

```

{
    switch (state) {
        case WIFI_CONNECTED:
            return "CONNECTED";
        case WIFI_CONNECTING:
            return "CONNECTING";
        case WIFI_DISCONNECTED:
            return "DISCONNECTED";
        default:
            return "UNKNOWN";
    }
}

static void WifiStaFormatMac(char *buf, size_t buf_len, const uint8_t *mac)
{
    if (buf == NULL || buf_len < 18 || mac == NULL) {
        return;
    }
    (void)snprintf(buf, buf_len, "%02X:%02X:%02X:%02X:%02X:%02X",
        mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
}

/**
 * @brief 打印当前关联 AP 的链路信息 (SSID/BSSID/RSSI 等)
 */
static void WifiStaPrintLinkInfo(const char *tag, const wifi_linked_info_stru
*info)
{
    char mac_str[18] = {0};

    if (info == NULL) {
        return;
    }
    WifiStaFormatMac(mac_str, sizeof(mac_str), (const uint8_t *)info->bssid);
    printf("[WifiStaDemo] %s SSID=%s BSSID=%s state=%s rssi=%d snr=%d ch=%d\r\n",
        tag, (const char *)info->ssid, mac_str,
        wifiStaConnStateStr(info->conn_state),
        (int)info->rssi, (int)info->snr, (int)info->channel_num);
}

/**
 * @brief STA 与 AP 关联状态变化回调 (由 Wi-Fi 服务在关联成功/失败后调用)
 * @param state      WIFI_STATE_AVALIABLE 表示链路可用, NOT_AVALIABLE 表示断开
 * @param info       当前链路信息, 可能为 NULL
 * @param reason_code 断开原因码, 具体含义见 SDK 文档
 */
static void WifiStaOnConnectionChanged(int32_t state, const wifi_linked_info_stru
*info, int32_t reason_code)
{
    if (state == WIFI_STATE_NOT_AVALIABLE) {
        g_sta_stats.disconnect++;
        printf("[WifiStaDemo] event disconnect #%u reason=%d\r\n",
            (unsigned)g_sta_stats.disconnect, (int)reason_code);
        wifiStaPrintLinkInfo(" ", info);
        return;
    }
}

```

```

        g_sta_stats.connect_ok++;
        printf("[WifiStaDemo] event connect #u\r\n",
(unsigned)g_sta_stats.connect_ok);
        WifiStaPrintLinkInfo(" ", info);
    }

/**
 * @brief 扫描结束通知; WifiStaConnectAp 内部会触发 wifi_sta_scan
 */
static void WifiStaOnScanStateChanged(int32_t state, int32_t size)
{
    g_sta_stats.scan_done++;
    printf("[WifiStaDemo] event scan_done #u state=%ld ap_count=%ld\r\n",
        (unsigned)g_sta_stats.scan_done, (long)state, (long)size);
}

/**
 * @brief 注册 STA 事件; 建议在 WifiStaConnectAp 之前调用以便收到全程事件
 */
static errcode_t WifiStaRegisterEvents(void)
{
    static wifi_event_stru s_ev;

    if (g_sta_event_registered) {
        return ERRCODE_SUCC;
    }

    (void)memset(&s_ev, 0, sizeof(s_ev));
    s_ev.wifi_event_connection_changed = WifiStaOnConnectionChanged;
    s_ev.wifi_event_scan_state_changed = WifiStaOnScanStateChanged;

    if (wifi_register_event_cb(&s_ev) != ERRCODE_SUCC) {
        printf("[WifiStaDemo] register event fail\r\n");
        return ERRCODE_FAIL;
    }
    g_sta_event_registered = 1;
    return ERRCODE_SUCC;
}

/**
 * @brief 打印累计统计 + 当前 AP 信息 + wlan0 IPv4 (DHCP 成功后应有 192.168.43.x)
 */
static void WifiStaDumpStats(void)
{
    wifi_linked_info_stru info = {0};
    struct netif *netif = netif_find(WIFI_STA_IFNAME);

    printf("[WifiStaDemo] stats scan=%u connect=%u disconnect=%u\r\n",
        (unsigned)g_sta_stats.scan_done,
        (unsigned)g_sta_stats.connect_ok,
        (unsigned)g_sta_stats.disconnect);

    if (wifi_sta_get_ap_info(&info) == ERRCODE_SUCC) {
        WifiStaPrintLinkInfo("current", &info);
    } else {

```

```

        printf("[WifiStaDemo] current: get_ap_info fail\r\n");
    }

    /* lwIP 2.x: IPv4 存放在 ip_addr.u_addr.ip4.addr, 主机字节序 */
    if (netif != NULL && netif->ip_addr.u_addr.ip4.addr != 0) {
        uint32_t ip = netif->ip_addr.u_addr.ip4.addr;
        printf("[WifiStaDemo] IPv4=%u.%u.%u.%u\r\n",
            (unsigned)(ip & 0xff),
            (unsigned)((ip >> 8) & 0xff),
            (unsigned)((ip >> 16) & 0xff),
            (unsigned)((ip >> 24) & 0xff));
    } else {
        printf("[WifiStaDemo] IPv4=(none)\r\n");
    }
}

/**
 * @brief STA 板主任务
 */
static void WifiStaDemoTask(void *arg)
{
    (void)arg;

    while (wifi_is_wifi_initd() == 0) {
        osDelay(100);
    }

    (void)memset(&g_sta_stats, 0, sizeof(g_sta_stats));
    if (WifiStaRegisterEvents() != ERRCODE_SUCC) {
        return;
    }

    /* 扫描、关联、DHCP 细节见 wifi_sta_sdk.c */
    if (WifiStaConnectAp(WIFI_STA_SSID, WIFI_STA_PSK) != ERRCODE_SUCC) {
        printf("[WifiStaDemo] connect failed\r\n");
        wifiStaDumpStats();
        return;
    }

    printf("[WifiStaDemo] connect success, periodic stats every %ums\r\n",
        (unsigned)WIFI_STA_STATS_PERIOD_MS);
    wifiStaDumpStats();

    for (;;) {
        osDelay(WIFI_STA_STATS_PERIOD_MS);
        wifiStaDumpStats();
    }
}

static void WifiStaDemoEntry(void)
{
    osThreadAttr_t attr = {
        .name = "WifiStaDemo",
        .stack_size = 0x2000,
        .priority = osPriorityNormal,
    };
};

```



```

        if (osThreadNew(WifiStaDemoTask, NULL, &attr) == NULL) {
            printf("[WifiStaDemo] create task fail\r\n");
        }
    }

APP_FEATURE_INIT(WifiStaDemoEntry);

```

7.10 SoftAP 入口 `wifi_softap_demo.c` (AP 板烧录)

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/wifi_softap_demo.c` 同步 (文档更新时以仓库为准)。

```

/*
 * Copyright (c) 2026 SZTU Open Source Organization.
 * 适用平台：九联星闪开发板，海思 WS63E 芯片。
 *
 * wifi_softap_demo.c - 双板实验 **AP 板** 应用入口
 *
 * 职责：
 *   1. 注册 SoftAP 相关 Wi-Fi 事件（STA 关联/离开），在回调中打印并累计统计；
 *   2. 调用 wifi_softap_sdk 的 WifiSoftApStart 开热点；
 *   3. 周期调用 wifi_softap_get_sta_list 查询当前在线 STA。
 *
 * 烧录：BUILD.gn 的 sources 只保留本文件，注释 wifi_sta_demo.c。
 * 事件与统计逻辑 intentionally 放在 demo，不放在 wifi_softap_sdk.c。
 */
#include <stdio.h>
#include <string.h>
#include "ohos_init.h"
#include "cmsis_os2.h"
#include "errcode.h"
#include "wifi_device.h"
#include "wifi_device_config.h"
#include "wifi_event.h"
#include "wifi_hotspot.h"
#include "wifi_softap_sdk.h"

/* 与 STA 板 wifi_sta_demo.c 中宏必须一致，否则 STA 无法关联 */
#define WIFI_AP_SSID "OHOS_AP"
#define WIFI_AP_PSK "123456789"
/* 2.4G 信道，须与文档 7.2.1 及国家码要求一致 */
#define WIFI_AP_CHANNEL 7
/* wifi_softap_get_sta_list 缓冲区容量，与 SDK WIFI_DEFAULT_MAX_NUM_STA(8) 对齐 */
#define WIFI_AP_MAX_STA 8
/* 主循环中打印在线列表的周期（毫秒） */
#define WIFI_AP_STATS_PERIOD_MS 5000

/**
 * @brief AP 板侧 STA 关联事件累计（仅统计次数，在线数以 get_sta_list 为准）
 */
typedef struct {
    uint32_t sta_join; /* wifi_event_softap_sta_join 触发次数 */
    uint32_t sta_leave; /* wifi_event_softap_sta_leave 触发次数 */
} WifiSoftApStaStats;

```

```

static wifiSoftApStaStats g_ap_stats;
static uint8_t g_ap_event_registered; /* 防止重复 wifi_register_event_cb */

/**
 * @brief 将 6 字节 MAC 格式化为 "AA:BB:CC:DD:EE:FF" 便于串口阅读
 */
static void WifiSoftApFormatMac(char *buf, size_t buf_len, const uint8_t *mac)
{
    if (buf == NULL || buf_len < 18 || mac == NULL) {
        return;
    }
    (void)snprintf(buf, buf_len, "%02X:%02X:%02X:%02X:%02X:%02X",
        mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);
}

/**
 * @brief 打印单台 STA 的 MAC、信号强度、协商速率
 * @param tag 日志前缀, 如 "event" / " "
 * @param info SDK 下发的 wifi_sta_info_stru, 可为 NULL
 */
static void WifiSoftApPrintStaInfo(const char *tag, const wifi_sta_info_stru
*info)
{
    char mac_str[18] = {0};

    if (info == NULL) {
        return;
    }
    WifiSoftApFormatMac(mac_str, sizeof(mac_str), info->mac_addr);
    printf("[WifiSoftApDemo] %s MAC=%s RSSI=%d rate=%ukbps\r\n",
        tag, mac_str, (int)info->rssi, (unsigned)info->best_rate);
}

/**
 * @brief 热点有 STA 完成关联时由 Wi-Fi 服务回调（非本线程直接调用）
 * 实现位于 soc_wifi_service_api.c → send_broadcast_sta_connected_for_ap
 */
static void WifiSoftApOnStaJoin(const wifi_sta_info_stru *info)
{
    g_ap_stats.sta_join++;
    printf("[WifiSoftApDemo] event sta_join #%u\r\n",
(unsigned)g_ap_stats.sta_join);
    WifiSoftApPrintStaInfo(" ", info);
}

/**
 * @brief STA 从热点断开时回调
 */
static void WifiSoftApOnStaLeave(const wifi_sta_info_stru *info)
{
    g_ap_stats.sta_leave++;
    printf("[WifiSoftApDemo] event sta_leave #%u\r\n",
(unsigned)g_ap_stats.sta_leave);
    WifiSoftApPrintStaInfo(" ", info);
}

```

```

/**
 * @brief SoftAP 整体状态变化（可用/不可用），一般用于调试
 */
static void WifiSoftApOnStateChanged(int32_t state)
{
    printf("[WifiSoftApDemo] event softap_state=%ld (%s)\r\n", (long)state,
        state == WIFI_STATE_AVALIABLE ? "available" : "not_available");
}

/**
 * @brief 向协议栈注册本 demo 的事件表；须在 STA 连入前完成
 * @return ERRCODE_SUCC 注册成功；失败则后续无法收到 join/leave
 */
static errcode_t WifiSoftApRegisterEvents(void)
{
    static wifi_event_stru s_ev;

    if (g_ap_event_registered) {
        return ERRCODE_SUCC;
    }

    (void)memset(&s_ev, 0, sizeof(s_ev));
    s_ev.wifi_event_softap_sta_join = WifiSoftApOnStaJoin;
    s_ev.wifi_event_softap_sta_leave = WifiSoftApOnStaLeave;
    s_ev.wifi_event_softap_state_changed = WifiSoftApOnStateChanged;

    if (wifi_register_event_cb(&s_ev) != ERRCODE_SUCC) {
        printf("[WifiSoftApDemo] register event fail\r\n");
        return ERRCODE_FAIL;
    }
    g_ap_event_registered = 1;
    return ERRCODE_SUCC;
}

/**
 * @brief 周期统计：打印累计 join/leave，并主动查询当前关联 STA 列表
 * 与事件回调互补——回调反映“变化瞬间”，本函数反映“当前快照”
 */
static void WifiSoftApDumpStats(void)
{
    wifi_sta_info_stru sta_list[WIFI_AP_MAX_STA];
    uint32_t sta_num = WIFI_AP_MAX_STA;
    char mac_str[18];

    (void)memset(sta_list, 0, sizeof(sta_list));

    printf("[WifiSoftApDemo] stats join=%u leave=%u\r\n",
        (unsigned)g_ap_stats.sta_join, (unsigned)g_ap_stats.sta_leave);

    if (wifi_is_softap_enabled() == 0) {
        printf("[WifiSoftApDemo] online=0 (SoftAP disabled)\r\n");
        return;
    }

    /* sta_num 入参为数组容量，出参为实际在线数量 */
    if (wifi_softap_get_sta_list(sta_list, &sta_num) != ERRCODE_SUCC) {

```

```

        printf("[WifiSoftApDemo] online=? (get_sta_list fail)\r\n");
        return;
    }

    printf("[WifiSoftApDemo] online=%u\r\n", (unsigned)sta_num);
    for (uint32_t i = 0; i < sta_num; i++) {
        WifiSoftApFormatMac(mac_str, sizeof(mac_str), sta_list[i].mac_addr);
        printf("[WifiSoftApDemo] [%u] MAC=%s RSSI=%d rate=%ukbps\r\n",
            (unsigned)(i + 1), mac_str,
            (int)sta_list[i].rssi, (unsigned)sta_list[i].best_rate);
    }
}

/**
 * @brief AP 板任务: 等 Wi-Fi 就绪 → 注册事件 → 开热点 → 周期打印统计
 */
static void WifiSoftApDemoTask(void *arg)
{
    (void)arg;

    /* 系统启动后协议栈异步初始化, 未就绪时不可操作 Wi-Fi */
    while (wifi_is_wifi_initd() == 0) {
        osDelay(100);
    }

    (void)memset(&g_ap_stats, 0, sizeof(g_ap_stats));
    if (WifiSoftApRegisterEvents() != ERRCODE_SUCC) {
        return;
    }

    /* 热点参数与 DHCP 见 wifi_softap_sdk.c */
    if (WifiSoftApStart(WIFI_AP_SSID, WIFI_AP_PSK,
        WIFI_SEC_TYPE_WPA2PSK, WIFI_AP_CHANNEL) != ERRCODE_SUCC) {
        printf("[WifiSoftApDemo] start failed\r\n");
        return;
    }

    printf("[WifiSoftApDemo] hotspot ready, stats every %ums\r\n",
        (unsigned)WIFI_AP_STATS_PERIOD_MS);
    WifiSoftApDumpStats();

    for (;;) {
        osDelay(WIFI_AP_STATS_PERIOD_MS);
        WifiSoftApDumpStats();
    }
}

/**
 * @brief 应用入口: 创建 SoftAP demo 线程 (APP_FEATURE_INIT 由启动框架调用)
 */
static void WifiSoftApDemoEntry(void)
{
    osThreadAttr_t attr = {
        .name = "WifiSoftApDemo",
        .stack_size = 0x2000,
        .priority = osPriorityNormal,
    };

```

```

};
if (osThreadNew(WifiSoftApDemoTask, NULL, &attr) == NULL) {
    printf("[WifiSoftApDemo] create task fail\r\n");
}
}

APP_FEATURE_INIT(WifiSoftApDemoEntry);

```

7.11 BUILD.gn (二选一编译入口)

与仓库 `applications/sample/wifi-iot/app/wifi_sdk_demo/BUILD.gn` **同步** (文档更新时以仓库为准)。

```

# Copyright (c) 2026 SZTU Open Source Organization.
# 适用平台: 九联星闪开发板, 海思 WS63E 芯片。
#
# 双板实验编译切换 (同一固件只能有一个 *_demo.c):
#   STA 板: 保留 wifi_sta_demo.c, 注释 wifi_softap_demo.c
#   AP 板: 保留 wifi_softap_demo.c, 注释 wifi_sta_demo.c
#
# 公共库: wifi_sta_sdk / wifi_softap_sdk / wifi_sdk_common 两板都会链接;
# 入口 demo 二选一, 避免两个 APP_FEATURE_INIT 同时运行。

static_library("wifi_sdk_demo") {
    sources = [
        "wifi_sdk_common.c",
        "wifi_sta_sdk.c",
        "wifi_softap_sdk.c",
        # "wifi_sta_demo.c",
        "wifi_softap_demo.c",
    ]

    defines = [ "LWIP_CONFIG_FILE=\"lwip/lwipopts_default.h\"" ]

    include_dirs = [
        "//commonlibrary/utils_lite/include",
        "//kernel/liteos_m/kal/cmsis",
        "//base/iothardware/peripheral/interfaces/inner_api",

        "//device/soc/hisilicon/ws63v100/sdk/open_source/lwip/lwip_v2.1.3/src/include",

        "//device/soc/hisilicon/ws63v100/sdk/open_source/lwip/lwip_adapter/liteos_207/src/include",

        "//device/soc/hisilicon/ws63v100/sdk/kernel/liteos/liteos_v208.5.0/Huawei_LiteOS/compat/linux/include",
        "//device/soc/hisilicon/ws63v100/sdk/kernel/osal/include/lock",
        "//device/soc/hisilicon/ws63v100/sdk/kernel/osal/include/schedule",
        "//device/soc/hisilicon/ws63v100/sdk/middleware/utils/common_headers/native",
        "//device/soc/hisilicon/ws63v100/sdk/kernel/osal/include",
        "//device/soc/hisilicon/ws63v100/sdk/include/middleware/services/wifi",
    ]
}

```

7.12 双板联调步骤（AP 板 + STA 板）

步骤	AP 板	STA 板
1	工程只编译 <code>wifi_softap_demo.c</code> ， <code>python3 applications/make_app.py -b</code>	工程只编译 <code>wifi_sta_demo.c</code> ，同样编译
2	烧录固件，USB 接串口，上电	确认 <code>WIFI_STA_SSID / PSK</code> 与 AP 板一致后烧录
3	串口出现 <code>started SSID=OHOS_AP、IP=192.168.43.1</code>	待 AP 板热点已开启 后再给 STA 板上电或复位
4	<code>[wifiSoftApDemo] event sta_join、stats ... online=1</code>	<code>event connect、scan round、[WIFI_STA] IP 192.168.43.x</code>
5	记录本板 IP <code>192.168.43.1</code>	记录本板 IP（如 <code>192.168.43.2</code> ），填写实验报告
6	串口 <code>[wifiTcpServer] listening</code> ，收到 <code>STA-MSG</code>	串口 <code>[wifiTcpClient] connected</code> ，收发 <code>AP-ACK</code>

验收：STA 板串口打印非 0 的 `192.168.43.x`，且与 AP 板参数一致；两板同时在线超过 1 分钟无掉线。

常见失误：STA 板先上电而 AP 板热点未开；两板 SSID/密码不一致；两板烧录了同一角色（都成 STA 或都成 AP）。

7.13 示例要点

要点	说明
依赖	仅 Wi-Fi SDK + lwIP
STA 接口	<code>wifiStaConnectAp / wifiStaDisconnectAp</code> （7.5 ~ 7.6）
SoftAP 接口	<code>wifiSoftApStart / wifiSoftApStop</code> （7.7 ~ 7.8）
扫描重试	<code>WIFI_STA_RETRY_MAX</code> 次；单次扫描后等待 <code>WIFI_STA_SCAN_WAIT_MS</code> （默认 5s）
源码同步	第七节 7.3 ~ 7.11 与仓库 <code>wifi_sdk_demo/</code> 目录一致，修改代码后请同步更新本文档
TCP	<code>wifi_tcp_server.c / wifi_tcp_client.c</code> ，默认端口 8888 ，参数见 <code>wifi_tcp_common.h</code>

八、联网后的业务开发（MQTT / TCP）

1. AP 板、STA 板各使用第七节 `wifi_sdk_demo`（可共享工程目录），分别编译对应固件并烧录。

2. 先完成 **7.12 双板联调**，确认 STA 板已获 IP。
3. 在 STA 板 `wifiStaDemoTask` 联网成功后，可参考 `20_mqtt_demo` 做 MQTT (Broker 可放在连入 OHOS_AP 的 PC 上)。
4. 编译: `python3 applications/make_app.py -b`；烧录见 [03-环境配置和系统编译烧录文档.md](#)。

与 [13-星闪物联网综合实验指导书](#) 中 **Wi-Fi 上云** 选题一致。

九、配置与安全注意

- **分工**：一块板只能烧录一种角色 (AP 或 STA)，两板固件切勿混用同一 `*_demo.c`。
- **加密**：双板实验使用 WPA2-PSK；`softap_config_stru.security_type` 与密码长度需匹配 SDK 要求。
- **信道**：AP 板 `WIFI_AP_CHANNEL` 取值与选型见 **7.2.1**；双板实验时信道由 AP 板决定，STA 板自动跟随所连 AP。
- **内存**：Wi-Fi + MQTT + 多任务同时开启时关注栈大小与 `malloc` 失败日志。
- **回调**：MQTT/Wi-Fi 事件回调中避免长时间阻塞；置标志位由任务处理 (与 GPIO 控制分离)。

十、常见问题

Q1: STA 板扫描不到 OHOS_AP 或一直连接失败?

- **先确认 AP 板已上电** 且串口已打印 `started SSID=OHOS_AP`。
- 两板 `SSID / 密码` 宏完全一致；STA 板编译的是 `wifi_sta_demo.c`，AP 板是 `wifi_softap_demo.c`。
- 将 STA 板 `WIFI_STA_SCAN_WAIT_MS` 加大到 **8000 ~ 10000**；缩短两板距离。
- 可选用 `wifi_event_scan_state_changed` 回调替代固定延时。

Q2: 双板已连上但 ping 不通?

- 使用 STA 串口打印的 IP 与 AP 的 `192.168.43.1` 互 ping；检查是否同网段 `192.168.43.x`。
- 若用 PC 参与，PC 需连接热点 `OHOS_AP` (可选，非双板实验必需)。

Q3: MQTT 连接失败?

- Broker 填 PC 在 `OHOS_AP` 网段 下的 IP，勿填 `127.0.0.1`；PC 须连入 AP 板热点或与 STA 同网段。

Q4: 串口反复 `wiFi has been initialized?`

- 原因：系统已初始化 Wi-Fi，应用再次 `while (wifi_init())` 会永远失败。请使用 **7.4 节** `wifi_sdk_common.c` 的 `WifiSdkEnsureInit()` (只尝试一次 `wifi_init`)。

Q5: `[WIFI_AP] dhcp_start fail?`

- 若同时出现 `[WIFI_STA]` 与 `[WIFI_AP]`：固件里 **STA/AP 两个 demo 都被编译**，请按 7.0 节只保留一个 `*_demo.c` 后重编烧录。

- 已修复：等待 `wifi_is_wifi_initied()`、`softap_enable` 后等待 `ap0`、网关改为 **192.168.43.2**（与 SDK `softap_sample` 一致）、`dhcps_start` 前 `dhcps_stop`、lwIP 返回值用 `ERR_OK` 判断。
- `srv:265:cd is not init` 为未注册 `wifi_event` 的提示，新版已注册空回调，一般不影响 DHCP。

Q6: Wi-Fi 与 SLE 同时用？

- 可以，注意初始化顺序与射频共存说明；综合实验见 [12-综合项目案例](#)、[13-星闪物联网综合实验指导书](#)。

十一、参考文档

资料	位置
WS63 系列用户指南	官方文档 README 中的 PDF 链接
WS63V100 lwIP 开发指南	<code>applications/docs/官方文档/docs/software/</code>
WS63V100 MQTT 开发指南	同上
WS63V100 HTTP 开发指南	同上
环境编译烧录	03-环境配置和系统编译烧录文档.md
综合实验 Wi-Fi 选题	12-综合项目案例.md 、 13-星闪物联网综合实验指导书.md
厂商 demo 总表	官方文档/README.md 案例教程一节

说明：接口名以海思 SDK 与 OpenHarmony 适配层为准，不同 SDK 版本可能有增减，升级后请对照 `wifi_device.h` 与 release note。